

GROUPE 5

Rapport

Application ArtConnect

**Aude LABAT
Thomas DURIAUD
Evan LADEIRA
Paul FONTAINE**

Etape 1 :

Comprendre ArtConnect et définir le périmètre fonctionnel

Objectif: Comprendre l'application fournie, définir votre vision fonctionnelle d'ArtConnect et fixer le périmètre exact que vous implémentez en base de données.

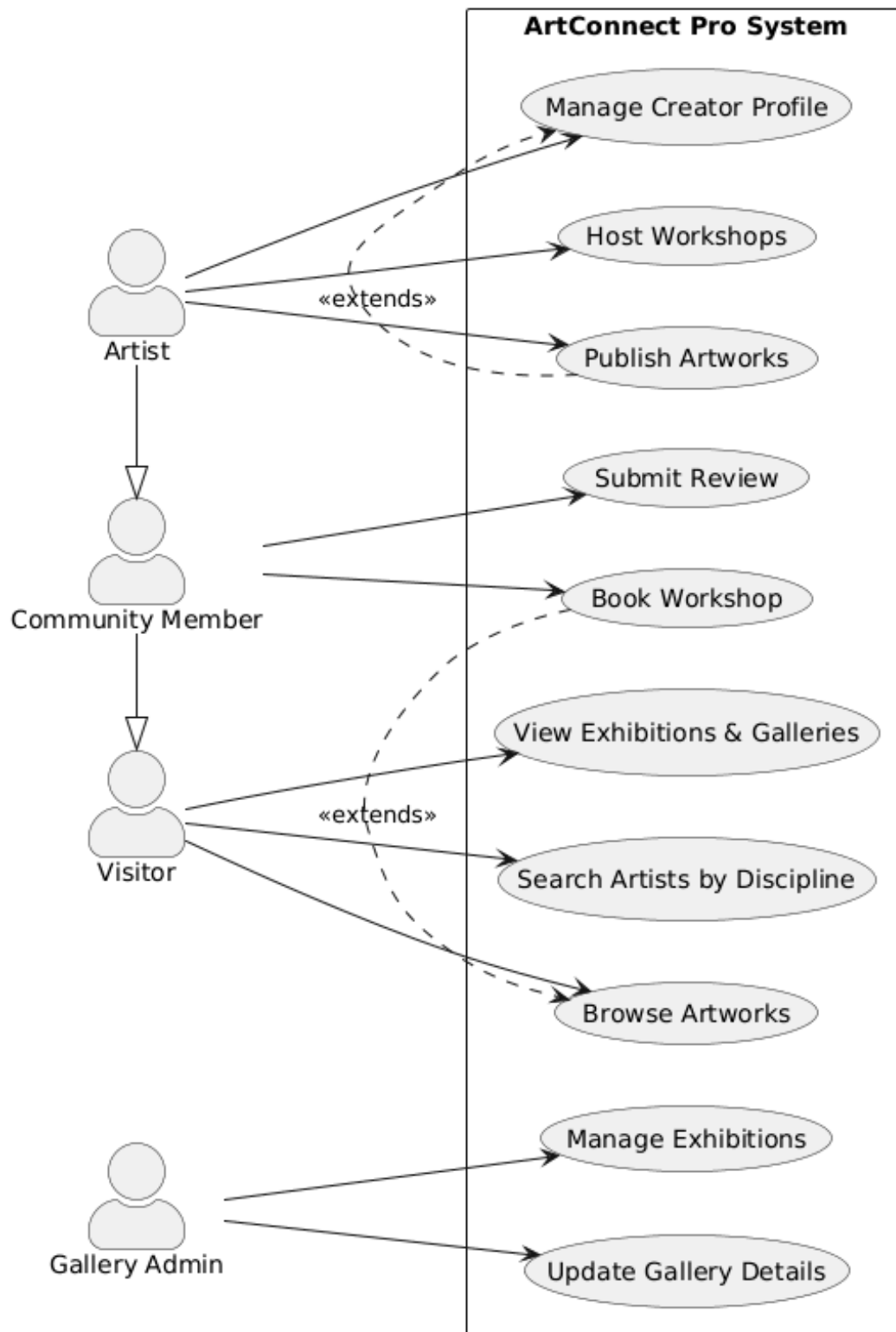
1 - Liste des fonctionnalités de l'application Java

- Exposer/mettre en avant les futures expositions et workshops
 - Lister les artistes et certaines informations personnelles (ville, mail, date d'anniversaire)
- > Dans cette dernière catégorie, l'utilisateur peut effectuer une recherche par nom d'artiste et filtrer ces derniers par discipline)
- Lister les œuvres et leurs informations relatives (auteur, prix, titre ...)
 - Lister les galeries et leur adresse
 - Lister les expositions et leurs informations relatives (titre, nom de la galerie, date de commencement ...)
 - Lister les workshops et leurs informations relatives (titre, nom de l'instructeur, dates, prix ...)
 - Lister les utilisateurs inscrits sur l'application et leurs informations de compte (nom, mail, ville)

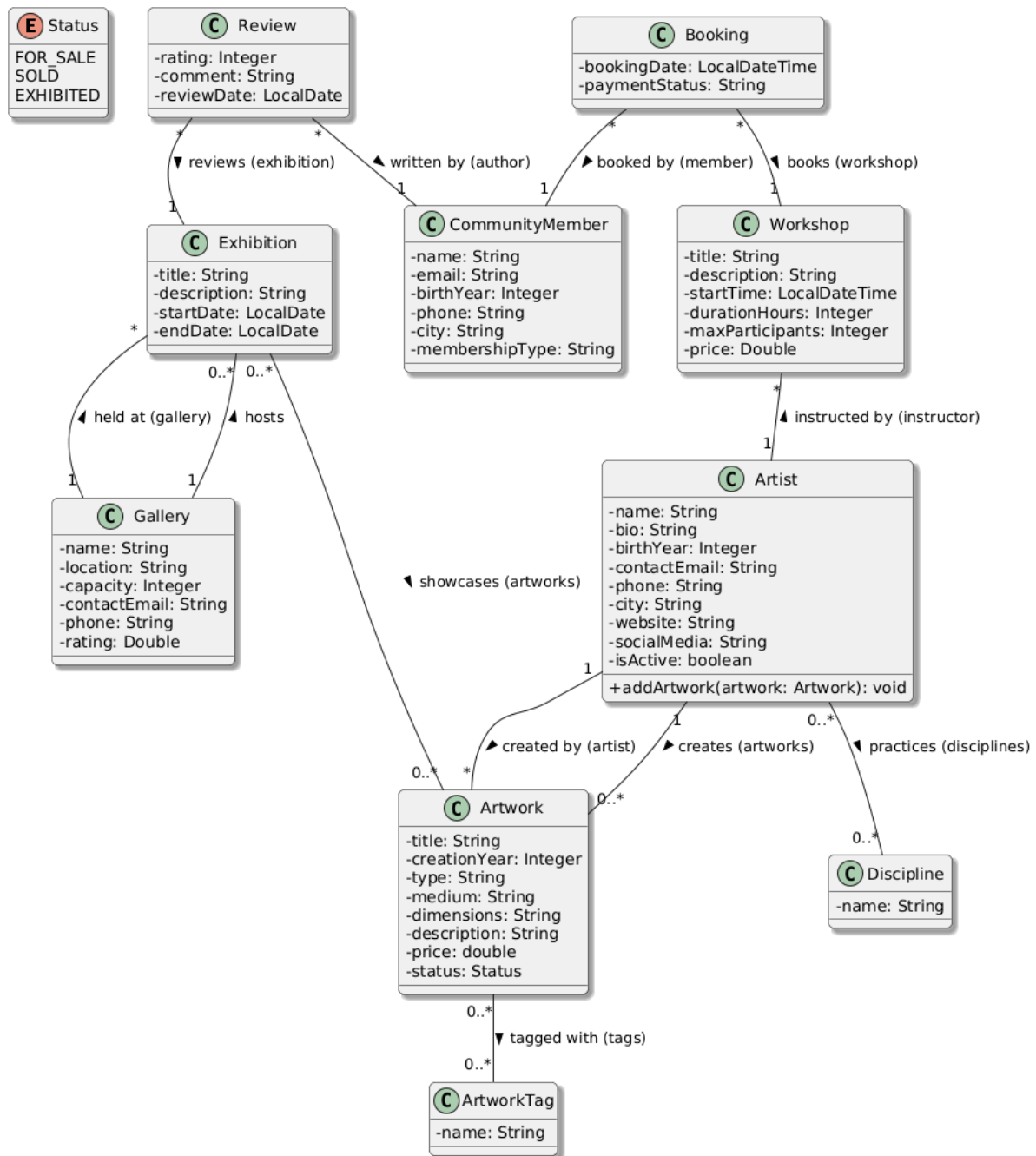
2 - Diagramme des cas d'utilisation

On peut identifier 4 utilisateurs distincts :

- **Visiteur** : accès en lecture seule (pas de compte). Peut parcourir les artistes, voir les œuvres et consulter les événements.
- **Membre** : visiteur mais avec les actions propres à la communauté : s'inscrire à un événement, suivre un artiste, accéder à la section Discover.
- **Artiste** : gère sa présence sur la plateforme : profil, publications d'œuvres, création/participation à des ateliers.
- **Organisateur** : administre les événements et les galeries.



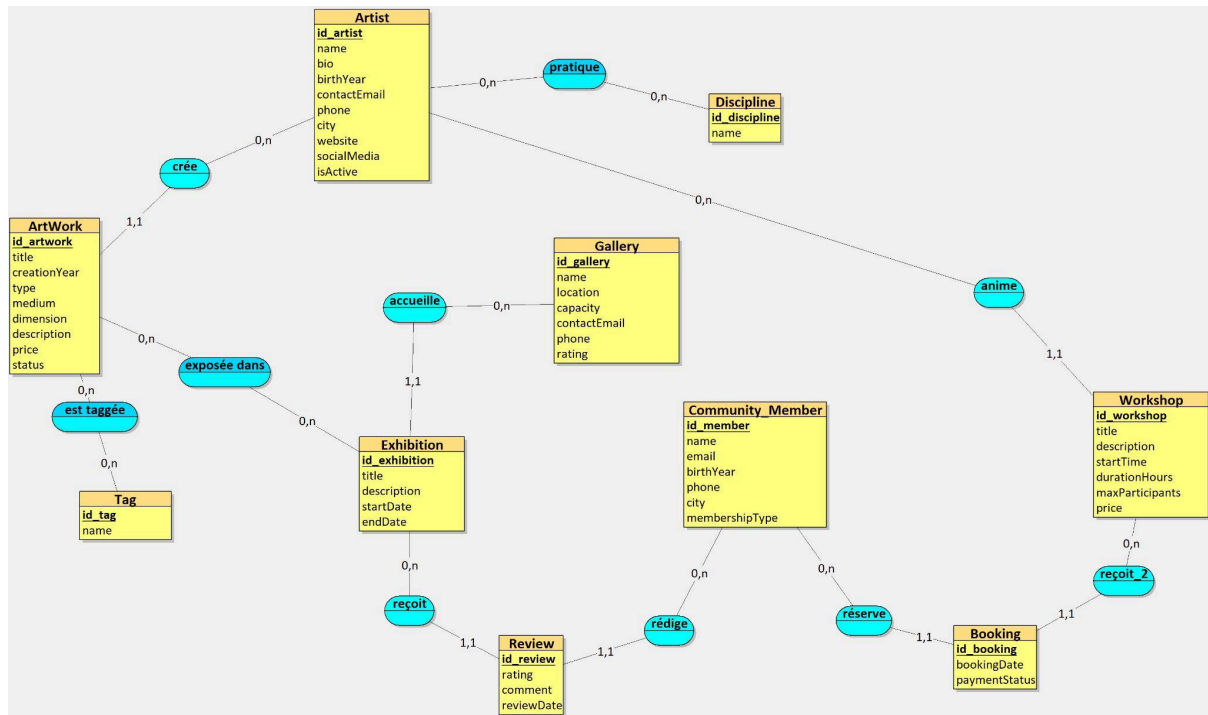
3- Diagramme de classes



Etape 2 : Modélisation conceptuelle et logique de la base ArtConnect

Objectif: Concevoir les modèles conceptuel et logique de la base de données à partir des fonctionnalités identifiées, en appliquant les principes de normalisation (jusqu'à la 3FN).

1 - Modèle Conceptuel de Données



2 - Modèle Logique de Données

```

MLD
Artist = (id_artist INT, name VARCHAR(50), bio TEXT, birthYear INT, contactEmail VARCHAR(50), phone VARCHAR(50), city VARCHAR(50), website VARCHAR(50), socialMedia VARCHAR(50), isActive LOGICAL);
ArtWork = (id_artwork INT, title VARCHAR(50), creationYear INT, type VARCHAR(50), medium VARCHAR(50), dimension VARCHAR(50), description TEXT, price VARCHAR(50), status VARCHAR(50), #id_artist);
Tag = (id_tag INT, name VARCHAR(50));
Gallery = (id_gallery INT, name VARCHAR(50), location VARCHAR(50), capacity INT, contactEmail VARCHAR(50), phone VARCHAR(50), rating DECIMAL(15,2));
Workshop = (id_workshop INT, title VARCHAR(50), description VARCHAR(200), startTime DATE, durationHours INT, maxParticipants INT, price DECIMAL(15,2), #id_artist);
Community_Member = (id_member INT, name VARCHAR(50), email VARCHAR(50), birthYear INT, phone VARCHAR(50), city VARCHAR(50), membershipType VARCHAR(50));
Discipline = (id_discipline INT, name VARCHAR(50));
Exhibition = (id_exhibition INT, title VARCHAR(50), description VARCHAR(200), startDate DATE, endDate DATE, #id_gallery);
Booking = (id_booking INT, bookingDate DATE, paymentStatus VARCHAR(50), #id_workshop, #id_member);
Review = (id_review INT, rating INT, comment TEXT, reviewDate DATE, #id_member, #id_exhibition);
est_taggée = (#id_artwork, #id_tag);
exposée_dans = (#id_artwork, #id_exhibition);
pratique = (#id_artist, #id_discipline);
    
```

3 - Explication des étapes de normalisation.

La normalisation est un processus qui permet d'organiser les données afin d'éliminer les redondances et les anomalies d'insertion, de modification ou de suppression. On applique successivement des formes normales.

Première Forme Normale (1FN)

Règle : chaque attribut doit contenir une valeur atomique (indivisible) et chaque ligne doit être unique.

Dans la conception initiale, plusieurs attributs posaient problème. L'attribut socialMedia dans ARTIST pouvait contenir plusieurs valeurs en même temps (ex : Instagram, Twitter, LinkedIn). De même, l'attribut tags dans ARTWORK pouvait regrouper plusieurs tags séparés par des virgules, et l'attribut disciplines dans ARTIST pouvait lister plusieurs disciplines en une seule cellule. Ces attributs multi-valués ne respectent pas la 1FN.

Pour y remédier, les tags ont été extraits dans une table dédiée TAG, reliée à ARTWORK via la table de liaison ARTWORK_TAG. De même, les disciplines ont été isolées dans une table DISCIPLINE, reliée à ARTIST via ARTIST_DISCIPLINE. L'attribut socialMedia a quant à lui été conservé comme champ unique de type VARCHAR, limitant sa valeur à une seule entrée.

Deuxième Forme Normale (2FN)

Règle : la table doit être en 1FN, et chaque attribut non-clé doit dépendre de la totalité de la clé primaire.

Après application de la 1FN, les tables de liaison ARTIST_DISCIPLINE, ARTWORK_TAG et ARTWORK_EXHIBITION possèdent des clés primaires composées. Si ces tables avaient contenu des attributs supplémentaires dépendant uniquement d'une partie de la clé, cela aurait constitué une violation de la 2FN. Par exemple, si ARTWORK_TAG avait contenu un attribut tagCategory qui ne dépendait que de id_tag et non de id_artwork, cela aurait été une dépendance partielle. Ce problème a été évité en ne conservant dans ces tables que les clés étrangères nécessaires, et en plaçant tout attribut descriptif directement dans la table concernée (ici TAG).

Troisième Forme Normale (3FN)

Règle : la table doit être en 2FN, et aucun attribut non-clé ne doit dépendre d'un autre attribut non-clé.

Dans la conception initiale, la table WORKSHOP contenait un attribut instructor de type texte, stockant directement le nom de l'intervenant. Cela impliquait que des informations liées à cet instructeur, comme sa ville ou son email, auraient dû être répétées dans chaque ligne de WORKSHOP où il intervenait. Ces attributs auraient dépendu de instructor et non de id_workshop, ce qui constitue une dépendance transitive et ne respecte pas la 3FN.

Pour corriger cela, l'attribut instructor a été remplacé par une clé étrangère id_artist pointant vers la table ARTIST. Ainsi, toutes les informations relatives à l'instructeur sont centralisées dans ARTIST, et WORKSHOP ne contient plus que la référence vers cette entité.

4 - Script SQL pour la création de la base de données

```
DROP DATABASE IF EXISTS ArtConnect;
```

```
CREATE DATABASE ArtConnect;
```

```
USE ArtConnect;
```

```
CREATE TABLE Artist (  
    artist_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    bio TEXT,  
    birthYear INT,  
    contactEmail VARCHAR(100),  
    phone VARCHAR(30),  
    city VARCHAR(100),  
    website VARCHAR(255),  
    socialMedia VARCHAR(255),  
    isActive BOOLEAN  
);
```

```
CREATE TABLE Discipline (  
    discipline_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Artist_Discipline (  
    artist_id INT,  
    discipline_id INT,  
    PRIMARY KEY (artist_id, discipline_id),  
    FOREIGN KEY (artist_id) REFERENCES Artist(artist_id),  
    FOREIGN KEY (discipline_id) REFERENCES Discipline(discipline_id)  
);
```

```
CREATE TABLE Artwork (  
    artwork_id INT AUTO_INCREMENT PRIMARY KEY,  
    artist_id INT NOT NULL,  
    title VARCHAR(200),  
    creationYear INT,  
    type VARCHAR(100),  
    medium VARCHAR(100),  
    dimensions VARCHAR(100),  
    description TEXT,  
    price DECIMAL(10,2),  
    status ENUM('FOR_SALE', 'SOLD', 'EXHIBITED'),  
    FOREIGN KEY (artist_id) REFERENCES Artist(artist_id)  
);
```

```
CREATE TABLE ArtworkTag (  
    artwork_id INT,  
    tag VARCHAR(100),  
    PRIMARY KEY (artwork_id, tag),  
    FOREIGN KEY (artwork_id) REFERENCES Artwork(artwork_id)
```

```
        tag_id INT AUTO_INCREMENT PRIMARY KEY,  
        name VARCHAR(100)  
    );
```

```
CREATE TABLE Artwork_Tag (  
    artwork_id INT,  
    tag_id INT,  
    PRIMARY KEY (artwork_id, tag_id),  
    FOREIGN KEY (artwork_id) REFERENCES Artwork(artwork_id),  
    FOREIGN KEY (tag_id) REFERENCES ArtworkTag(tag_id)  
);
```

```
CREATE TABLE Gallery (  
    gallery_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(150),  
    location VARCHAR(255),  
    capacity INT,  
    contactEmail VARCHAR(100),  
    phone VARCHAR(30),  
    rating DOUBLE  
);
```

```
CREATE TABLE Exhibition (  
    exhibition_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200),  
    description TEXT,  
    startDate DATE,  
    endDate DATE,  
    gallery_id INT,  
    FOREIGN KEY (gallery_id) REFERENCES Gallery(gallery_id)  
);
```

```
CREATE TABLE Artwork_Exhibition (  
    artwork_id INT,  
    exhibition_id INT,  
    PRIMARY KEY (artwork_id, exhibition_id),  
    FOREIGN KEY (artwork_id) REFERENCES Artwork(artwork_id),  
    FOREIGN KEY (exhibition_id) REFERENCES Exhibition(exhibition_id)  
);
```

```
CREATE TABLE CommunityMember (  
    member_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    birthYear INT,  
    phone VARCHAR(30),  
    city VARCHAR(100),  
    membershipType VARCHAR(50)
```

);

```
CREATE TABLE Review (  
    review_id INT AUTO_INCREMENT PRIMARY KEY,  
    rating INT CHECK (rating BETWEEN 1 AND 5),  
    comment TEXT,  
    reviewDate DATE,  
    member_id INT,  
    exhibition_id INT,  
    FOREIGN KEY (member_id) REFERENCES CommunityMember(member_id),  
    FOREIGN KEY (exhibition_id) REFERENCES Exhibition(exhibition_id)  
);
```

```
CREATE TABLE Workshop (  
    workshop_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(200),  
    description TEXT,  
    startTime DATETIME,  
    durationHours INT,  
    maxParticipants INT,  
    price DECIMAL(10,2),  
    instructor_id INT,  
    FOREIGN KEY (instructor_id) REFERENCES Artist(artist_id)  
);
```

```
CREATE TABLE Booking (  
    booking_id INT AUTO_INCREMENT PRIMARY KEY,  
    bookingDate DATETIME,  
    paymentStatus VARCHAR(50),  
    member_id INT,  
    workshop_id INT,  
    FOREIGN KEY (member_id) REFERENCES CommunityMember(member_id),  
    FOREIGN KEY (workshop_id) REFERENCES Workshop(workshop_id)  
);
```

Etape 3 : Implémentation de la Base de Données et Fonctionnalités Avancées

Objectif: Créer la base dans le SGBD, l'alimenter avec des données d'exemple et mettre en place les objets BD avancés (vues, index, triggers, procédures/fonctions stockées).

1 - Prompt utilisé pour le LLM pour insérer des données d'exemple

Voici le prompt utilisé pour générer, avec un LLM, des données d'exemple :

“

Génère un script d'insertion MySQL pour la base de données ArtConnect. J'ai besoin de données fictives mais réalistes pour démontrer l'application. Insère : 4 Disciplines, 5 ArtworkTags, 3 Galeries, 4 Artistes, 3 CommunityMembers. Ensuite, génère 6 Œuvres (Artwork) réparties entre les artistes, 2 Expositions rattachées aux galeries et contenant quelques œuvres, 2 Ateliers (Workshop) animés par les artistes, 4 Réservations (Booking) de membres pour ces ateliers, et 3 Avis (Review). Utilise des IDs explicites ou des sous-requêtes si nécessaire pour respecter les clés étrangères. Assure-toi que les dates sont cohérentes (ex: date de réservation avant la date de l'atelier).

Pour t'orienter dans la création de ce script, voici le code SQL utilisé pour créer la base de données :

code SQL de la partie 4 du livrable n°2

“

2 - Script d'insertion généré

```
USE ArtConnect;
```

```
-- 1. Insertion des Artistes
```

```
INSERT INTO Artist (name, bio, birthYear, contactEmail, phone, city, website, socialMedia, isActive) VALUES ('Elena Rostova', 'Artiste peintre contemporaine.', 1985, 'elena@art.com', '0601020304', 'Paris', 'elenart.com', '@elena_art', TRUE), ('Marcus Dubois', 'Sculpteur métal et bois.', 1978, 'marcus@metal.com', '0611223344', 'Lyon', 'marcus-sculpt.com', '@marcus_d', TRUE), ('Sophie Lemoine', 'Photographe minimaliste.', 1990, 'sophie@photo.com', '0622334455', 'Bordeaux', 'sophiephoto.fr', '@sophie_l', TRUE), ('Julien Clerc', 'Artiste pluridisciplinaire', 1982, 'julien@create.com', '0633445566', 'Paris', NULL, NULL, FALSE);
```

```
-- 2. Insertion des Disciplines
```

```
INSERT INTO Discipline (name) VALUES ('Peinture'), ('Sculpture'),
```

```

('Photographie');

-- 3. Liaison Artist_Discipline
INSERT INTO Artist_Discipline (artist_id, discipline_id) VALUES (1, 1),
(2, 2), (3, 3), (4, 1), (4, 2);

-- 4. Insertion des Œuvres
INSERT INTO Artwork (artist_id, title, creationYear, type, medium,
dimensions, description, price, status) VALUES
(1, 'Aube Parisienne', 2023, 'Peinture', 'Huile', '100x80cm', 'Une vue
de Paris à l aube.', 1200.00, 'FOR_SALE'),
(1, 'Reflets', 2021, 'Peinture', 'Aquarelle', '50x50cm', 'Jeu de reflets
sur l eau.', 800.00, 'SOLD'),
(2, 'Structure 01', 2024, 'Sculpture', 'Acier', '200x100x100cm',
'Sculpture monumentale.', 3500.00, 'EXHIBITED'),
(2, 'Équilibre', 2022, 'Sculpture', 'Bois et Métal', '40x30x30cm',
'Petite sculpture de bureau.', 650.00, 'FOR_SALE'),
(3, 'Ombres Urbaines', 2023, 'Photographie', 'Argentique', '60x40cm',
'Tirage noir et blanc.', 450.00, 'FOR_SALE'),
(3, 'Silence', 2024, 'Photographie', 'Numérique', '90x60cm', 'Paysage
minimaliste dans la neige.', 550.00, 'EXHIBITED');

-- 5. Insertion des Tags
INSERT INTO ArtworkTag (name) VALUES ('Abstrait'), ('Paysage'),
('Contemporain'), ('Noir et Blanc'), ('Monumental');

-- 6. Liaison Artwork_Tag
INSERT INTO Artwork_Tag (artwork_id, tag_id) VALUES (1, 2), (1, 3), (3,
3), (3, 5), (5, 4), (6, 2), (6, 4);

-- 7. Insertion des Galeries
INSERT INTO Gallery (name, location, capacity, contactEmail, phone,
rating) VALUES
('Galerie Horizon', '12 Rue des Arts, Paris', 150, 'contact@horizon.fr',
'0140506070', 4.8),
('Lumière Noire', '8 Place Bellecour, Lyon', 80,
'hello@lumierenoire.com', '0478809010', 4.5);

-- 8. Insertion des Expositions
INSERT INTO Exhibition (title, description, startDate, endDate,
gallery_id) VALUES
('Visions Contemporaines', 'Les nouveaux talents de la scène
française.', '2026-05-01', '2026-06-15', 1),
('Matière et Lumière', 'Exposition autour de la sculpture et de la
photographie.', '2026-07-10', '2026-08-20', 2);

```

```

-- 9. Liaison Artwork_Exhibition
INSERT INTO Artwork_Exhibition (artwork_id, exhibition_id) VALUES (1,
1), (3, 2), (5, 2), (6, 2);

-- 10. Insertion des Membres de la communauté
INSERT INTO CommunityMember (name, email, birthYear, phone, city,
membershipType) VALUES
('Alice Dubois', 'alice@mail.com', 1995, '0699887766', 'Paris',
'Premium'),
('Lucas Martin', 'lucas@mail.com', 1988, '0688776655', 'Lyon',
'Standard'),
('Emma Petit', 'emma@mail.com', 1992, '0677665544', 'Nantes',
'Standard');

-- 11. Insertion des Reviews (Livrées sur les Expositions dans ton
modèle)
INSERT INTO Review (rating, comment, reviewDate, member_id,
exhibition_id) VALUES
(5, 'Une exposition incroyable, très bien agencée.', '2026-05-10', 1,
1),
(4, 'Belles œuvres mais un peu trop de monde le week-end.',
'2026-07-15', 2, 2);

-- 12. Insertion des Ateliers (Workshops)
INSERT INTO Workshop (title, description, startTime, durationHours,
maxParticipants, price, instructor_id) VALUES
('Initiation à la Peinture à l huile', 'Découvrez les bases avec
Elena.', '2026-06-05 14:00:00', 3, 10, 45.00, 1),
('Photographie Urbaine de Nuit', 'Sortie photo dans les rues.',
'2026-06-12 21:00:00', 4, 8, 60.00, 3);

-- 13. Insertion des Réservations (Bookings)
INSERT INTO Booking (bookingDate, paymentStatus, member_id, workshop_id)
VALUES
('2026-05-20 10:00:00', 'PAID', 1, 1),
('2026-05-21 11:30:00', 'PAID', 2, 1),
('2026-05-22 15:45:00', 'PENDING', 3, 1),
('2026-06-01 09:00:00', 'PAID', 1, 2);

```

=> Ayant été généré par un LLM, on vérifie le script produit sur une version de test de la database sur MySQL WorkBench. On a ainsi le résultat suivant :

```

1 • USE ArtConnect;
2
3 -- 1. Insertion des Artistes
4 INSERT INTO Artist (name, bio, birthYear, contactEmail, phone, city, website, socialMedia, isActive) VALUES
5 ('Elena Rostova', 'Artiste peintre contemporaine.', 1985, 'elena@art.com', '0601020304', 'Paris', 'elenart.com', '@elena_art', TRUE),
6 ('Marcus Dubois', 'Sculpteur métal et bois.', 1978, 'marcus@metal.com', '0611223344', 'Lyon', 'marcus-sculpt.com', '@marcus_d', TRUE),
7 ('Sophie Lemoine', 'Photographe minimaliste.', 1990, 'sophie@photo.com', '0622334455', 'Bordeaux', 'sophiephoto.fr', '@sophie_1', TRUE),
8 ('Julien Clerc', 'Artiste pluridisciplinaire', 1982, 'julien@create.com', '0633445566', 'Paris', NULL, NULL, FALSE);
9
10 -- 2. Insertion des Disciplines
11 INSERT INTO Discipline (name) VALUES ('Peinture'), ('Sculpture'), ('Photographie');
12
13 -- 3. Liaison Artist_Discipline
14 INSERT INTO Artist_Discipline (artist_id, discipline_id) VALUES (1, 1), (2, 2), (3, 3), (4, 1), (4, 2);
15
16 -- 4. Insertion des Œuvres
17 INSERT INTO Artwork (artist_id, title, creationYear, type, medium, dimensions, description, price, status) VALUES
18 (1, 'Aube Parisienne', 2023, 'Peinture', 'Huile', '100x80cm', 'Une vue de Paris à l'aube.', 1200.00, 'FOR_SALE'),
19 (1, 'Reflets', 2021, 'Peinture', 'Aquarelle', '50x50cm', 'Jeu de reflets sur l'eau.', 800.00, 'SOLD'),
20 (2, 'Structure 01', 2024, 'Sculpture', 'Acier', '200x100x100cm', 'Sculpture monumentale.', 3500.00, 'EXHIBITED'),
21 (2, 'Équilibre', 2022, 'Sculpture', 'Bois et Métal', '40x30x30cm', 'Petite sculpture de bureau.', 650.00, 'FOR_SALE'),
22 (3, 'Ombres Urbaines', 2023, 'Photographie', 'Argentique', '60x40cm', 'Tirage noir et blanc.', 450.00, 'FOR_SALE'),
23 (3, 'Silence', 2024, 'Photographie', 'Numérique', '90x60cm', 'Paysage minimaliste dans la neige.', 550.00, 'EXHIBITED');
24
25 -- 5. Insertion des Tags
26

```

#	Time	Action	Message	Duration / Fetch
62	11:16:07	INSERT INTO Gallery (name, location, capacity, contactEmail, phone, rating) VALUES (Galerie Horizon, '12...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
63	11:16:07	INSERT INTO Exhibition (title, description, startDate, endDate, gallery_id) VALUES (Visions Contemporaines...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
64	11:16:07	INSERT INTO Artwork_Exhibition (artwork_id, exhibition_id) VALUES (1, 1), (3, 2), (5, 2), (6, 2)	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.015 sec
65	11:16:07	INSERT INTO CommunityMember (name, email, birthYear, phone, city, membershipType) VALUES (Alice Du...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
66	11:16:07	INSERT INTO Review (rating, comment, reviewDate, member_id, exhibition_id) VALUES (5, 'Une exposition ...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
67	11:16:07	INSERT INTO Workshop (title, description, startTime, durationHours, maxParticipants, price, instructor_id) V...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
68	11:16:07	INSERT INTO Booking (bookingDate, paymentStatus, member_id, workshop_id) VALUES (2026-05-20 10.0...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.015 sec

-> L'entièreté du script produit s'est donc exécutée avec succès.

3 - Documentation des déclencheurs, procédures stockées et vues

A. Vues

Objectif : Faciliter l'affichage de la boutique. Ne montre que les œuvres à vendre d'artistes actifs, en joignant le nom de l'artiste pour éviter les jointures répétitives côté Java.

```

CREATE VIEW Active_Artworks_For_Sale AS
SELECT a.artwork_id, a.title, a.price, a.medium, art.name AS
artist_name, art.city
FROM Artwork a
JOIN Artist art ON a.artist_id = art.artist_id
WHERE a.status = 'FOR_SALE' AND art.isActive = TRUE;

```

Objectif : Calculer le nombre de places restantes pour chaque atelier à la volé

```

CREATE VIEW Workshop_Availability AS
SELECT w.workshop_id, w.title, w.startTime, w.maxParticipants,
COUNT(b.booking_id) AS currentBookings,
(w.maxParticipants - COUNT(b.booking_id)) AS availableSeats
FROM Workshop w
LEFT JOIN Booking b ON w.workshop_id = b.workshop_id

```

```
GROUP BY w.workshop_id, w.title, w.startTime, w.maxParticipants;
```

Objectif : Agréger les données d'une exposition et de sa galerie d'accueil, c'est utile pour la page des expositions en cours.

```
CREATE VIEW VW_Exhibition_Details AS
SELECT e.exhibition_id, e.title AS exhibition_title, e.startDate,
e.endDate,
       g.name AS gallery_name, g.location AS gallery_location
FROM Exhibition e
JOIN Gallery g ON e.gallery_id = g.gallery_id;
```

B. Indexes

```
CREATE INDEX idx_artwork_status ON Artwork(status);
```

L'index est utile pour la vue Active_Artworks_For_Sale

```
CREATE INDEX idx_artist_name ON Artist(name);
```

C'est utile car les noms vont souvent être recherché, ça optimise donc ces requêtes

```
CREATE INDEX idx_workshop_start ON Workshop(startTime);
```

Ça permet de trier les ateliers par ordre des plus récents à venir.

C. Déclencheurs

On vérifie que l'atelier ne se finit pas avant d'avoir commencé :

```
CREATE TRIGGER Check_Dates_Exhibition
BEFORE INSERT ON Exhibition
FOR EACH ROW
BEGIN
    IF NEW.endDate < NEW.startDate THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La date de fin ne peut pas être antérieure à
la date de début.';
    END IF;
END //
```

On vérifie qu'il reste des places avant d'en vendre pour éviter le sur booking :

```
CREATE TRIGGER Prevent_Overbooking
```

```

BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    DECLARE v_currentBookings INT;
    DECLARE v_maxParticipants INT;

    SELECT COUNT(*) INTO v_currentBookings FROM Booking WHERE
workshop_id = NEW.workshop_id;
    SELECT maxParticipants INTO v_maxParticipants FROM Workshop WHERE
workshop_id = NEW.workshop_id;

    IF v_currentBookings >= v_maxParticipants THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Impossible de réserver : cet atelier est
complet.';
    END IF;
END //

```

On vérifie que chaque nouvelle réservation insérée est bien associée à une date (dans le cas contraire, on lui associe la date de 'maintenant')

```

CREATE TRIGGER TRG_Set_Booking_Date
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    IF NEW.bookingDate IS NULL THEN
        SET NEW.bookingDate = NOW();
    END IF;
END //

```

D. Procédure et fonctions stockées

Retourne exactement le nombre de places restantes pour un atelier donné.

```

CREATE FUNCTION FN_Get_Remaining_Seats(p_workshop_id INT)
RETURNS INT READS SQL DATA
BEGIN
    DECLARE v_max INT;
    DECLARE v_booked INT;

    SELECT maxParticipants INTO v_max FROM Workshop WHERE workshop_id =
p_workshop_id;
    SELECT COUNT(*) INTO v_booked FROM Booking WHERE workshop_id =
p_workshop_id;

    RETURN (v_max - v_booked);

```

END

Routine appelée quand un client de l'application Java valide un paiement pour une œuvre

```
CREATE PROCEDURE SP_Update_Artwork_Status(IN p_artwork_id INT, IN
p_new_status VARCHAR(50))
BEGIN
    UPDATE Artwork
    SET status = p_new_status
    WHERE artwork_id = p_artwork_id;
END
```

4 - Script de test des transactions

Un membre VIP (Alice Dubois, id=1) repère une œuvre à la vente (Aube Parisienne, id=1). Elle l'achète et décide de fêter ça en s'inscrivant à un atelier donné par n'importe quel artiste (ex: id=2).

Pour que cette suite d'opérations soit valide, il ne faut ni que l'œuvre ait été vendue entre-temps par un autre membre, ni que l'atelier soit complet.

```
START TRANSACTION;
```

```
-- On utilise des variables pour gérer le rollback manuel
```

```
SET @v_member_id = 1;
SET @v_artwork_id = 4;
SET @v_workshop_id = 2;
SET @v_artwork_status = '';
```

```
SELECT status INTO @v_artwork_status
FROM Artwork
WHERE artwork_id = @v_artwork_id FOR UPDATE;
```

```
IF @v_artwork_status != 'FOR_SALE' THEN
    ROLLBACK;
```

```
ELSE
```

```
    UPDATE Artwork SET status = 'SOLD' WHERE artwork_id = @v_artwork_id;
    INSERT INTO Booking (bookingDate, paymentStatus, member_id,
workshop_id)
    VALUES (NOW(), 'PAID', @v_member_id, @v_workshop_id);
    COMMIT;
END IF;
```

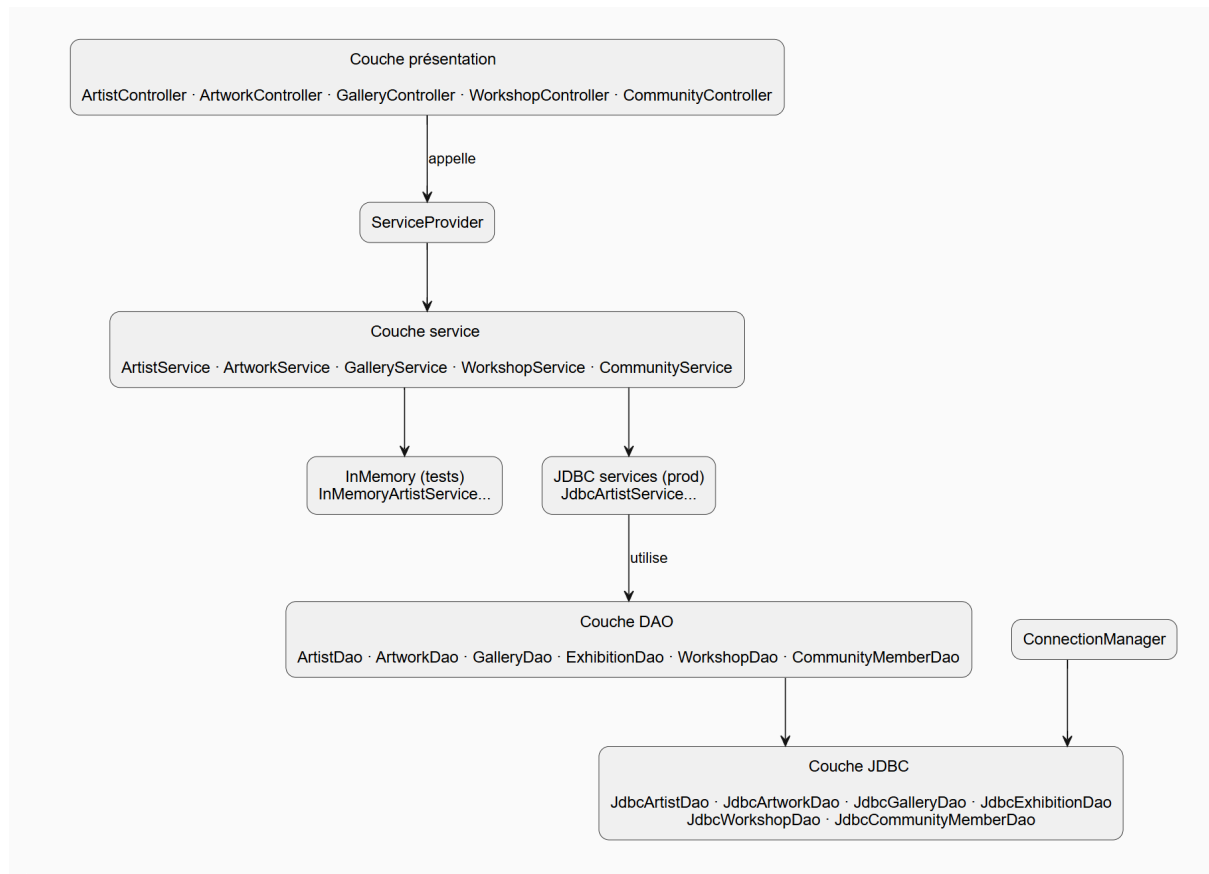
Etape 4 : Intégration de la base dans l'application Java ArtConnect

Objectif: Remplacer progressivement les services “en mémoire” de l'application ArtConnect par une couche de persistance réelle connectée à votre base ArtConnect.

1 - Code des entités, DAO et services.

lien github : <https://github.com/alalab12/ArtConnectPro-App>

2 - Description (courte) de l'architecture retenue (diagramme de classes simplifié ou schéma de couches).



L'application est structurée selon une architecture en quatre couches distinctes : présentation, service, DAO et persistance.

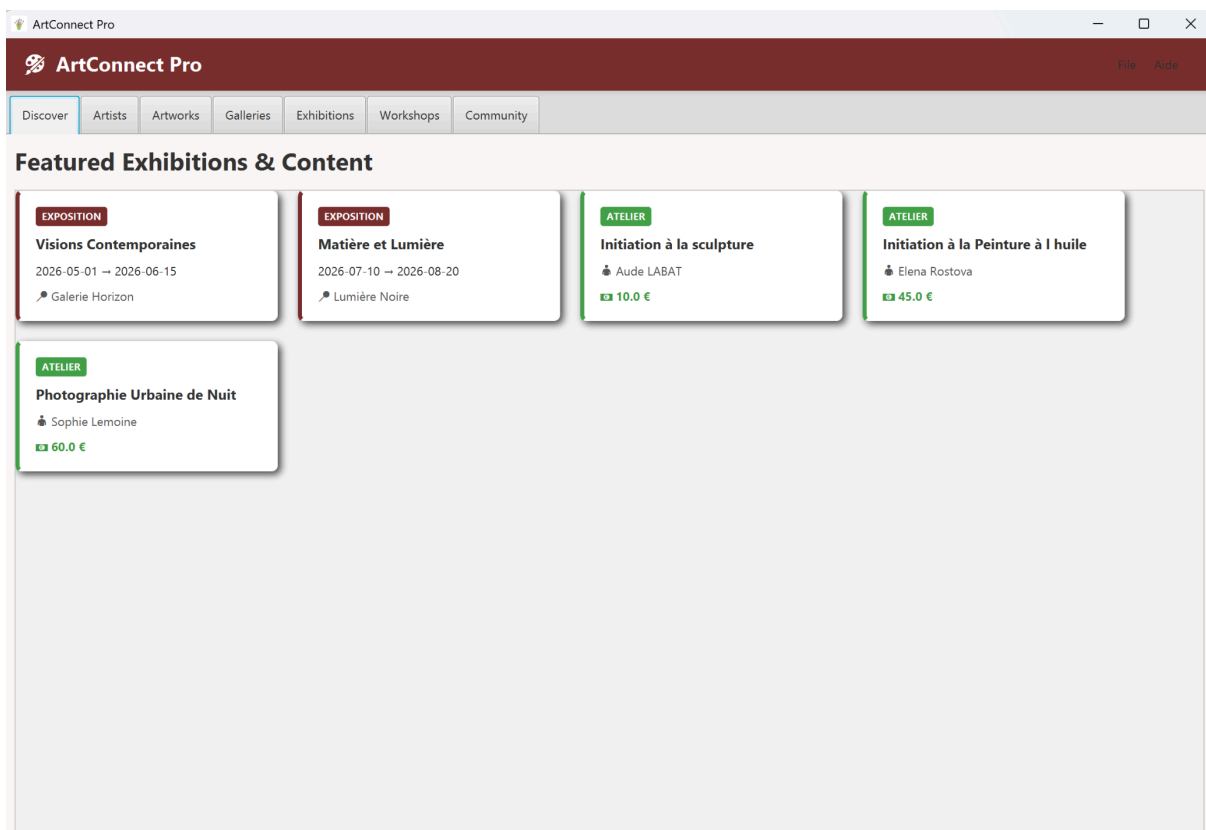
La couche de présentation (JavaFX) contient les contrôleurs, qui interagissent uniquement avec les interfaces de service. Ils sont indépendants du mode de stockage, ce qui a permis de passer à JDBC sans modification.

La couche service sert à mettre la logique métier au milieu de l'application : elle fait le lien entre l'interface utilisateur et l'accès aux données .

La couche DAO agit comme un contrat entre l'application et le stockage des données. Le Service demande des données au DAO, et le DAO délègue le travail technique à la couche JDBC.

JDBC (Java Database Connectivity) est l'API standard en Java qui fait office de "traducteur" entre l'application la base de données relationnelle MySQL.

3 - Captures d'écran ou vidéo de l'application ArtConnect en fonctionnement avec votre base.





 Modifier un artiste ✕

Nom *

Aude LABAT

Ville

Brest

Email

aude@gmail.com

Année

2005

Tél.

Bio

Enregistrer

Annuler

 Confirmation ✕

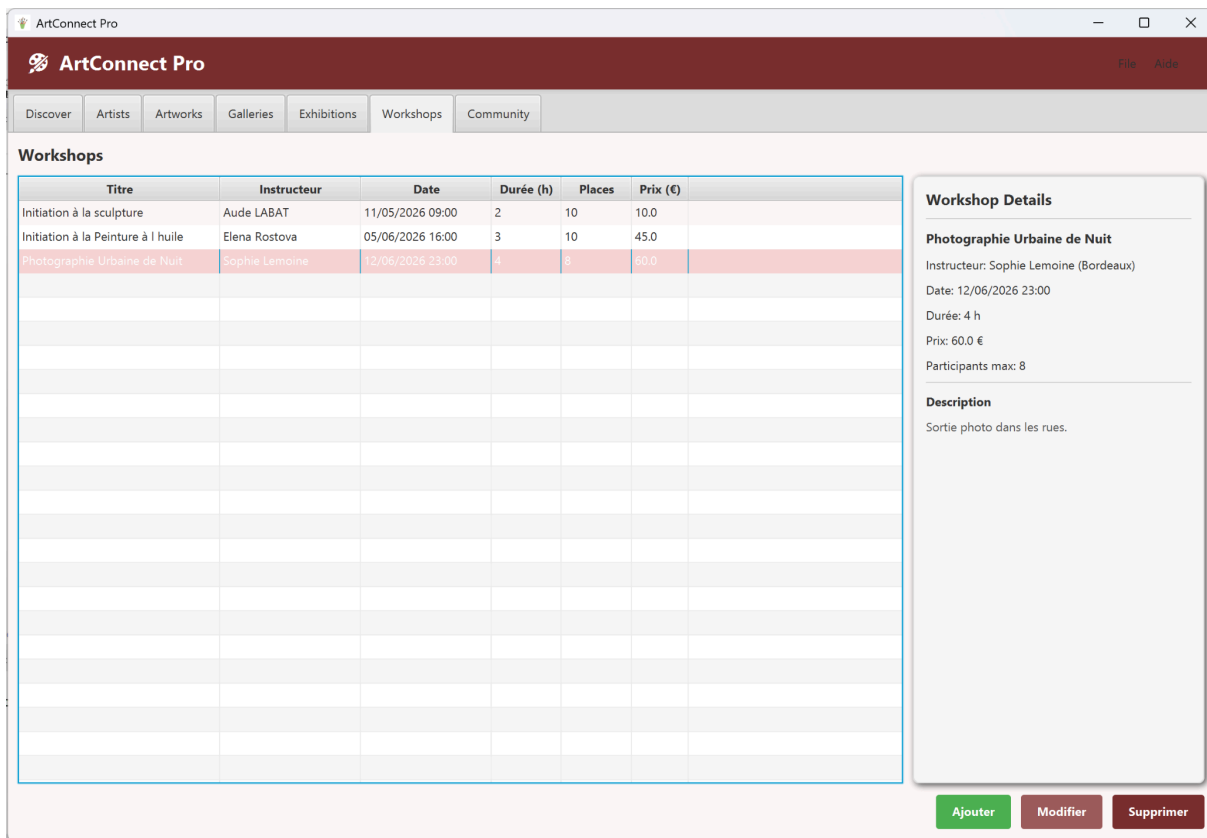
Confirmation



Supprimer "Aude LABAT" ?

OK

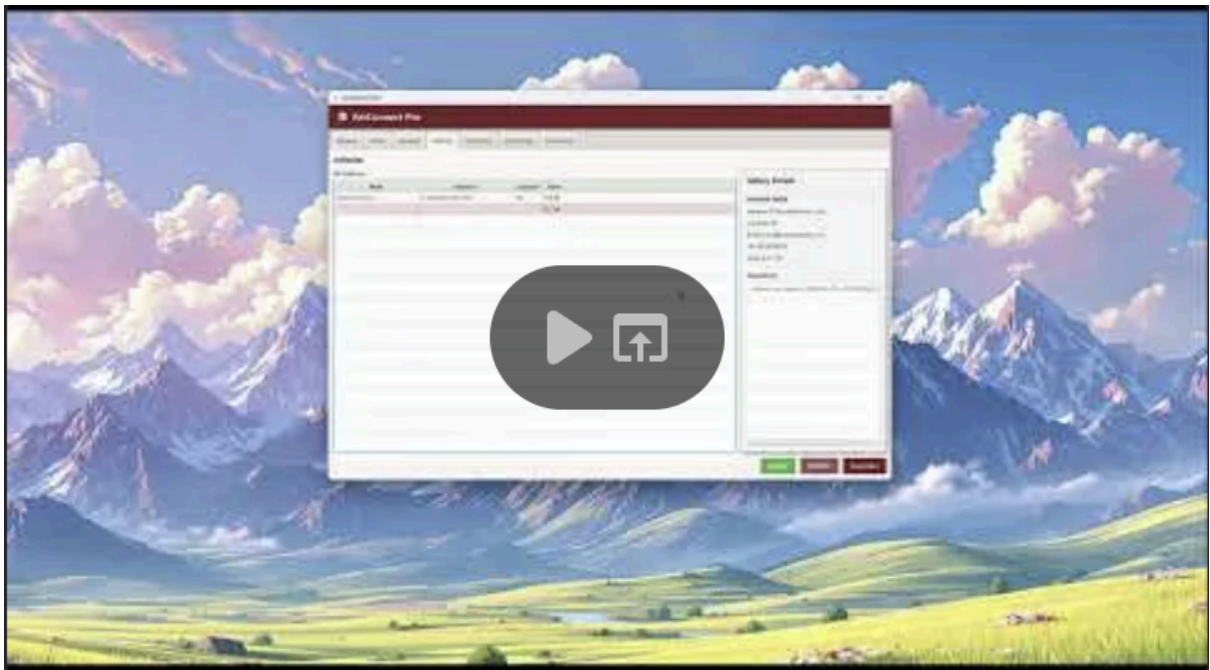
Annuler



Etape 5 : Finalisation et Présentation

Objectif: Finaliser le projet et préparer la présentation finale.

Notre vidéo de présentation du projet :



cliquez sur l'image pour voir la vidéo

Diapositives de la présentation du projet en cours ...